

# Writeup: Mission AlphaZeroOK

## Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup\_08\_Mission\_AlphaZeroOK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `code{Vyb#Hq6y?e}`

## Résumé

Write-up :

Contexte

On dispose du binaire ELF Mission\_Lancement\_Alpha\_Zero (64 bits).

L'Objectif : retrouver le flag de 16 caractères.

```
file Mission_Lancement_Alpha_Zero
```

## Outils Utilisés

Python scripte :

```
from pathlib import Path
```

```
data = Path('Mission_Lancement_Alpha_Zero(2)').read_bytes()
```

```
offset = 10499 + 14 #Calcul de l'offset des bloc après "Bad Password!" :
```

```
bloc1 = data[offset : offset+16]
```

```
bloc2 = data[offset+16 : offset+32]
```

```
bloc3 = data[offset+32 : offset+48]
```

```
flag = bytes([a ^ c for a, c in zip(bloc1, bloc3)])
```

```
print(flag.decode())
```

résultat : `rcode{Vyb#Hq6y?e`

16 octets, entièrement imprimables c'est sûrement le bon flag à quelques changements près.

Après avoir récupéré les 16 premiers caractères du flag en bloc1  $\oplus$  bloc3, nous avons constaté qu'il manquait la dernière accolade } pour fermer correctement le format. Il faut également retiré le caractère 'r' tout au début.

Étant donné que la consigne du challenge précise que le flag fait 16 caractères, il est logique de tester en complétant avec une accolade fermante } pour valider l'ensemble.

good flag : `code{Vyb#Hq6y?e}`

## Analyse Statique

Trois blocs de 16 octets juste à la suite : intéressant !

Vue rapide du code – objdump

```
objdump -d Mission_Lancement_Alpha_Zero | less
```

## Analyse Dynamique

L'exécution du programme a permis d'observer son comportement et d'identifier les points de validation.

## Identification du Mécanisme de Validation

Pas besoin de tout creuser:

on voit quatre petites fonctions appelées après la lecture de l'entrée ;

la 1ère vérifie la longueur == 16,

les 3 autres manipulent un tampon (XOR / rotations) et comparent aux 3 blocs.

Hypothèse & test dans Python

On soupçonne que deux blocs servent de clé/chiffrement pour le troisième (ou pour l'entrée).

Après avoir extrait deux blocs de 16 octets chacun dans la section .data du binaire, on a appliqué un XOR entre ces deux blocs (bloc1  $\oplus$  bloc3).

Cette opération est réversible et permet ici de retrouver directement le flag caché.

En effet, l'étudiant ayant développé le crackme avait stocké une version chiffrée du flag à l'aide de cette méthode simple mais efficace.

## Découverte du Flag

Un simple lancement sans argument :

```
./Mission_Lancement_Alpha_Zero
```

[ ... gros ASCII-art ... ]

La fusée Astraeus IX est prête...

Bad Password!

1. Repérage visuel – strings + xxd

```
strings -td Mission_Lancement_Alpha_Zero | grep "Bad Password"
```

10499 Bad Password!

On veut voir ce qu'il y a après "Bad Password!" :

```
xxd -s $((10499 + 14)) -l 64 'Mission_Lancement_Alpha_Zero(2)'
```

```
00002911: 007c cc33 acf3 b444 9348 fc3f e1eb 3556  .|.3...D.H?...5V
00002921: 206c 606b 6a74 3a6c 4840 3a75 7f2e 6479  l`kjt:IH@:u..dy
00002931: 721f a357 c988 e23d f16b b44e d792 0a33  r..W...=.k.N...3
00002941: 5d2c 0000 0002 0000 0000 0008 0000 0000  ],.....
```

## Conclusion

Après avoir extrait deux blocs de 16 octets chacun dans la section `.data` du binaire, on a appliqué un XOR entre ces deux blocs (bloc1  $\oplus$  bloc3). Cette opération est réversible et permet ici de retrouver directement le flag caché. En effet, l'étudiant ayant développé le crackme avait stocké une version chiffrée du flag à l'aide de cette méthode simple mais efficace.

## Mission\_Lancement\_Alpha\_Zero

Mission\_Lancement\_Alpha\_Zero: ELF 64-bit LSB executable, x86-64, ...

```
chmod +x Mission_Lancement_Alpha_Zero
```